UNIT-3 (SORTING AND SEARCH TECHNIQUES)

SORTING ALGORITHMS: BASIC CONCEPTS, BUBBLE SORT

LECTURE NO:-17

Sorting is the process of arranging the elements of an array so that they can be placed either in ascending or descending order

SN	Sorting Algorithms	Description
1	Bubble Sort	It is the simplest sort method which performs sorting by repeatedly moving the largest element to the highest index of the array. It comprises of comparing each element to its adjacent element and replace them accordingly.
2	Bucket Sort	Bucket sort is also known as bin sort. It works by distributing the element into the array also called buckets. In this sorting algorithms, Buckets are sorted individually by using different sorting algorithm.
3	Comb Sort	Comb Sort is the advanced form of Bubble Sort. Bubble Sort compares all the adjacent values while comb sort removes all the turtle values or small values near the end of the list.
4	Counting Sort	It is a sorting technique based on the keys i.e. objects are collected according to keys which are small integers. Counting sort calculates the number of occurrence of objects and stores its key values. New array is formed by adding previous key elements and assigning to objects.
5	Heap Sort	In the heap sort, Min heap or max heap is maintained from the array elements depending upon the choice and the elements are sorted by deleting the root element of the heap.
6	Insertion Sort	As the name suggests, insertion sort inserts each element of the array to its proper place. It is a very simple sort method which is used to

Notes Prepared By: - Arvind Sharma (Assistant Professor, Department of CSE, MITRC Alwar)

		arrange the deck of cards while playing bridge.
7	Merge Sort	Merge sort follows divide and conquer approach in which, the list is first divided into the sets of equal elements and then each half of the list is sorted by using merge sort. The sorted list is combined again to form an elementary sorted array.
8	Quick Sort	Quick sort is the most optimized sort algorithms which performs sorting in O(n log n) comparisons. Like Merge sort, quick sort also work by using divide and conquer approach.
9	Radix Sort	In Radix sort, the sorting is done as we do sort the names according to their alphabetical order. It is the lenear sorting algorithm used for Inegers.
10	Selection Sort	Selection sort finds the smallest element in the array and place it on the first place on the list, then it finds the second smallest element in the array and place it on the second place. This process continues until all the elements are moved to their correct ordering. It carries running time $O(n^2)$ which is worst than insertion sort.
11	Shell Sort	Shell sort is the generalization of insertion sort which overcomes the drawbacks of insertion sort by comparing elements separated by a gap of several positions.

Bubble Sort is the simplest sorting algorithm that works by repeatedly swapping the adjacent elements if they are in the wrong order. This algorithm is not suitable for large data sets as its average and worst-case time complexity is quite high.

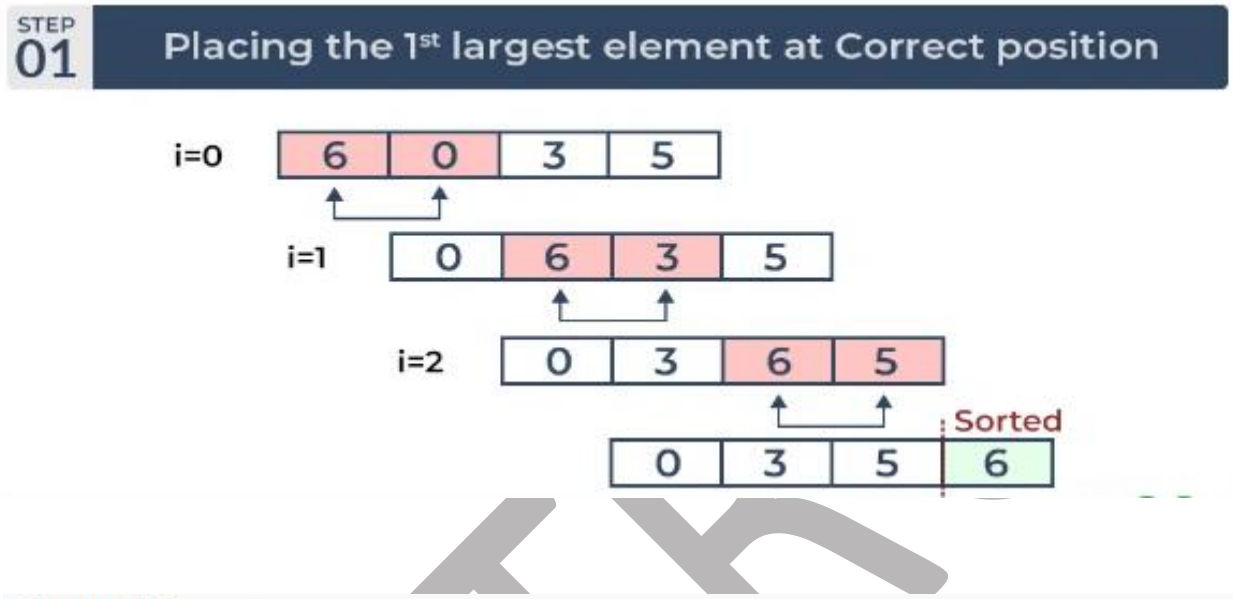
Algorithm

- traverse from left and compare adjacent elements and the higher one is placed at right side.
- In this way, the largest element is moved to the rightmost end at first.
- This process is then continued to find the second largest and place it and so on until the data is sorted.

Example

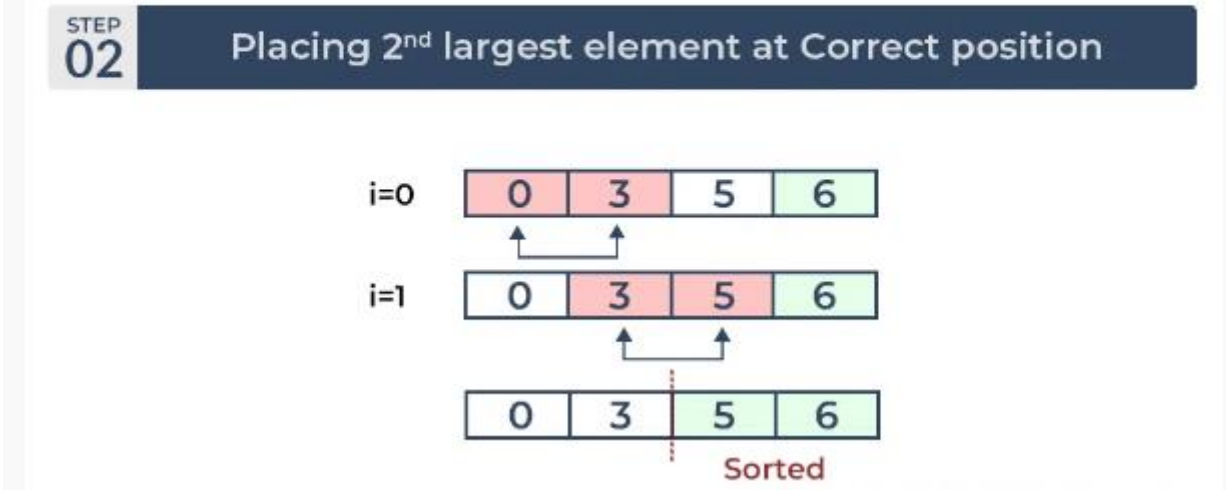
Input: arr[] = {6, 0, 3, 5}

First Pass:
The largest element is placed in its correct position, i.e., the end of the array.



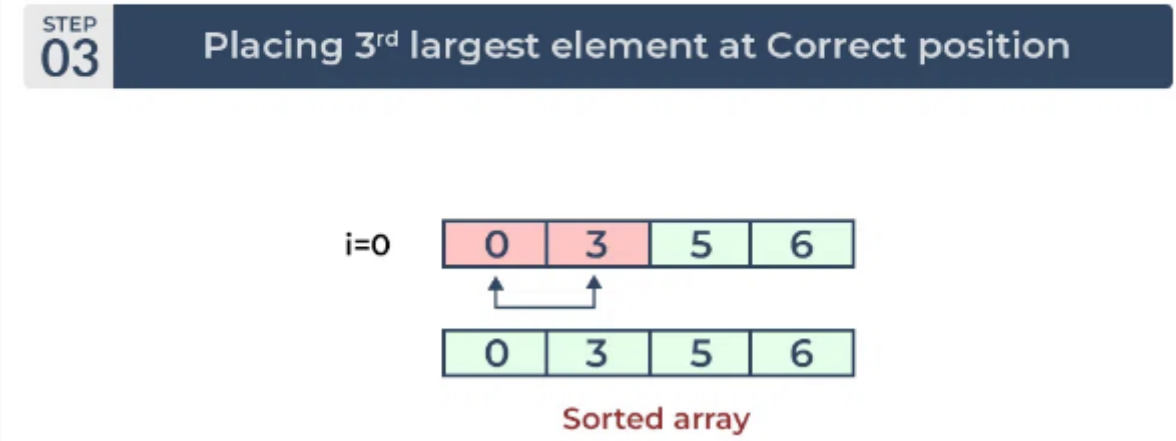
Second Pass:

Place the second largest element at correct position



Third Pass:

Place the remaining two elements at their correct positions.



Advantages of Bubble Sort:

- Bubble sort is easy to understand and implement.
- It does not require any additional memory space.
- It is a stable sorting algorithm, meaning that elements with the same key value maintain their relative order in the sorted output.

Disadvantages of Bubble Sort:

- Bubble sort has a time complexity of $O(N^2)$ which makes it very slow for large data sets.
- Bubble sort is a comparison-based sorting algorithm, which means that it requires a comparison operator to determine the relative order of elements in the input data set. It can limit the efficiency of the algorithm in certain cases.

UNIT-3 (SORTING AND SEARCH TECHNIQUES)

INSERTION SORT, SELECTION SORT

LECTURE NO:-18

Insertion sort is a simple sorting algorithm that works similar to the way you sort playing cards in your hands. The array is virtually split into a sorted and an unsorted part. Values from the unsorted part are picked and placed at the correct position in the sorted part.

Working of Insertion Sort algorithm

Consider an example: arr[]: **{12, 11, 13, 5, 6}**

12	**11**	**13**	**5**	**6**

First Pass:

- Initially, the first two elements of the array are compared in insertion sort.

12	**11**	13	5	6

- Here, 12 is greater than 11 hence they are not in the ascending order and 12 is not at its correct position. Thus, swap 11 and 12.
- So, for now 11 is stored in a sorted sub-array.

11	**12**	13	5	6

Second Pass:

- Now, move to the next two elements and compare them

11	**12**	**13**	5	6

- Here, 13 is greater than 12, thus both elements seems to be in ascending order, hence, no swapping will occur. 12 also stored in a sorted sub-array along with 11

Third Pass:

- Now, two elements are present in the sorted sub-array which are **11** and **12**
- Moving forward to the next two elements which are 13 and 5

Notes Prepared By: - Arvind Sharma (Assistant Professor, Department of CSE, MITRC Alwar)

11	12	**13**	**5**	6

- Both 5 and 13 are not present at their correct place so swap them

11	12	**5**	**13**	6

- After swapping, elements 12 and 5 are not sorted, thus swap again

11	**5**	**12**	13	6

- Here, again 11 and 5 are not sorted, hence swap again

5	**11**	12	13	6

- Here, 5 is at its correct position

Fourth Pass:

- Now, the elements which are present in the sorted sub-array are **5, 11** and **12**
- Moving to the next two elements 13 and 6

5	11	12	**13**	**6**

- Clearly, they are not sorted, thus perform swap between both

5	11	12	**6**	**13**

- Now, 6 is smaller than 12, hence, swap again

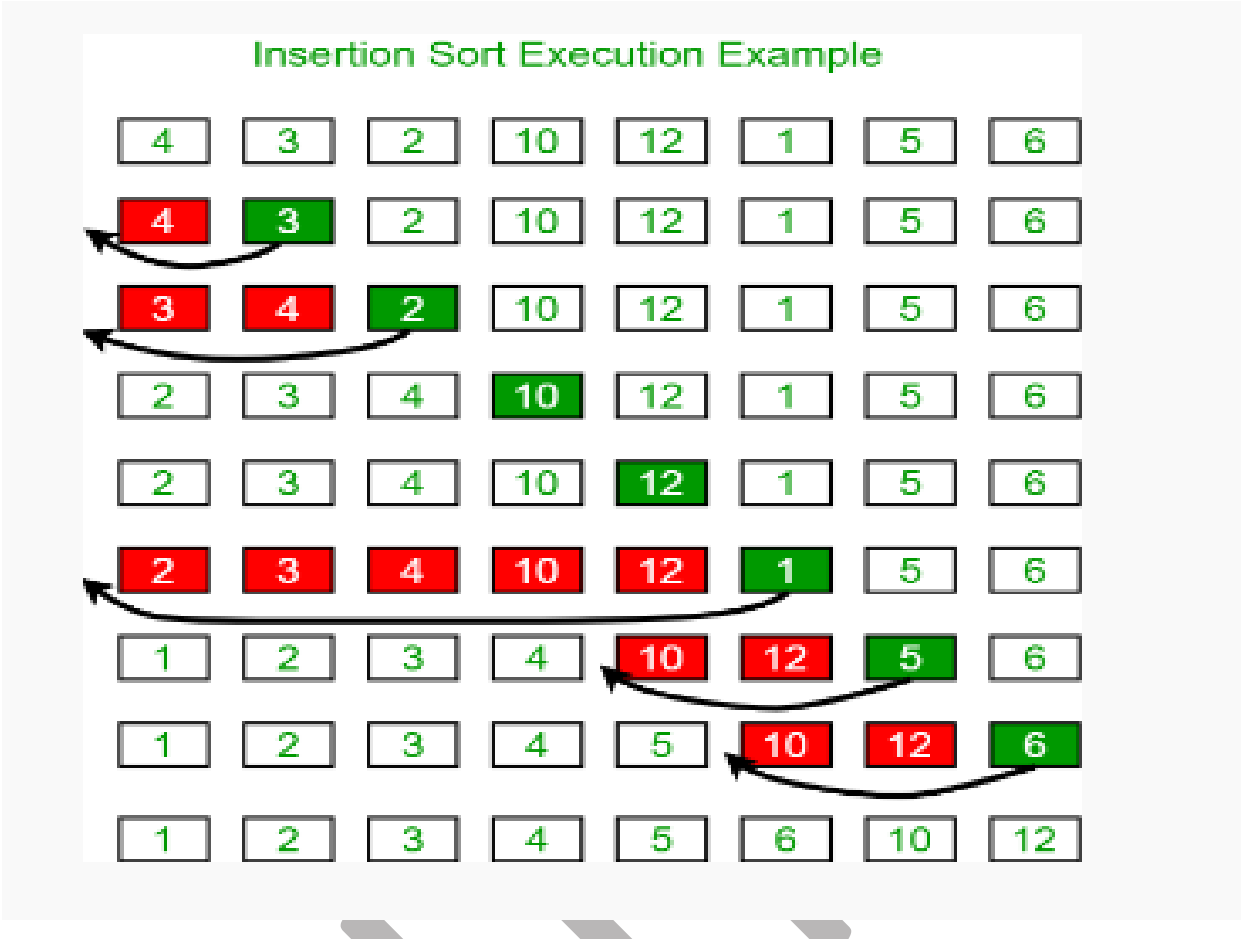
5	11	**6**	**12**	13

- Here, also swapping makes 11 and 6 unsorted hence, swap again

5	**6**	**11**	12	13

- Finally, the array is completely sorted.

MITRC



Algorithm

The simple steps of achieving the insertion sort are listed as follows -

Step 1 - If the element is the first element, assume that it is already sorted. Return 1.

Step 2 - Pick the next element, and store it separately in a **key.**

Step 3 - Now, compare the **key** with all elements in the sorted array.

Step 4 - If the element in the sorted array is smaller than the current element, then move to the next element. Else, shift greater elements in the array towards the right.

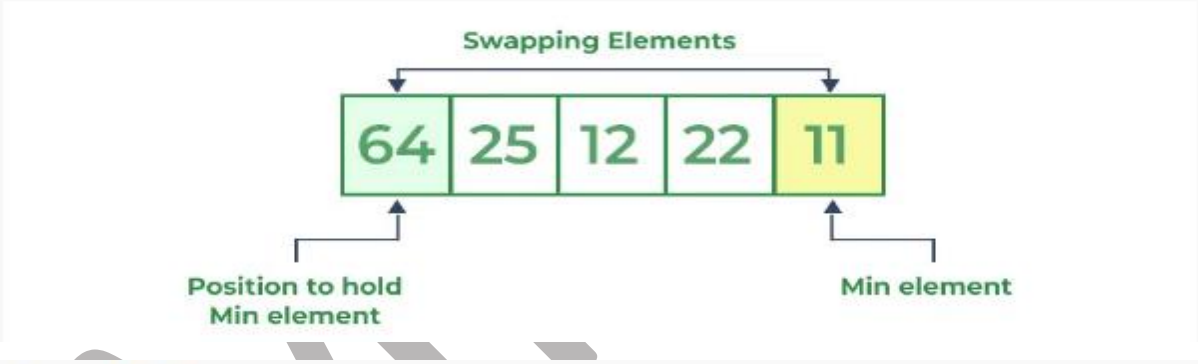
Step 5 - Insert the value.

Step 6 - Repeat until the array is sorted.

Selection sort *is a simple and efficient sorting algorithm that works by repeatedly selecting the smallest (or largest) element from the unsorted portion of the list and moving it to the sorted portion of the list.* The algorithm repeatedly selects the smallest (or largest) element from the unsorted portion of the list and swaps it with the first element of the unsorted part. This process is repeated for the remaining unsorted portion until the entire list is sorted.

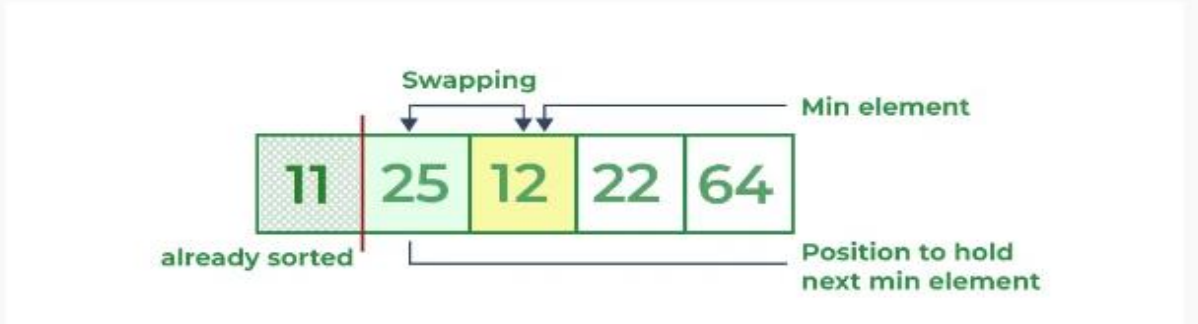
Working of Selection Sort

- *For the first position in the sorted array, the whole array is traversed from index 0 to 4 sequentially. The first position where **64** is stored presently, after traversing whole array it is clear that **11** is the lowest value.*
- *Thus, replace 64 with 11. After one iteration **11**, which happens to be the least value in the array, tends to appear in the first position of the sorted list.*



Second Pass:

- *For the second position, where 25 is present, again traverse the rest of the array in a sequential manner.*
- *After traversing, we found that **12** is the second lowest value in the array and it should appear at the second place in the array, thus swap these values.*



Third Pass:

- *Now, for third place, where **25** is present again traverse the rest of the array and find the third least value present in the array.*
- *While traversing, **22** came out to be the third least value and it should appear at the third place in the array, thus swap **22** with element present at third position.*



Fourth pass:

- *Similarly, for fourth position traverse the rest of the array and find the fourth least element in the array*
- *As **25** is the 4th lowest value hence, it will place at the fourth position.*



Fifth Pass:

- *At last the largest value present in the array automatically get placed at the last position in the array*
- *The resulted array is the sorted array.*



```
procedure selection sort
  list  : array of items
  n     : size of list
  for i = 1 to n - 1
  /* set current element as minimum*/
    min = i
    /* check the element to be minimum */
    for j = i+1 to n
      if list[j] < list[min] then
        min = j;
      end if
    end for
    /* swap the minimum element with the current element*/
    if indexMin != i  then
      swap list[min] and list[i]
    end if
  end for
end procedure
```

Advantages of Selection Sort Algorithm

- Simple and easy to understand.
- Works well with small datasets.

Disadvantages of the Selection Sort Algorithm

- Selection sort has a time complexity of O(n^2) in the worst and average case.
- Does not work well on large datasets.
- Does not preserve the relative order of items with equal keys which means it is not stable.

UNIT-3 (SORTING AND SEARCH TECHNIQUES)

QUICK SORT

LECTURE NO:-19

Quick Sort is a Divide and Conquer algorithm. It picks an element as a pivot and partitions the given array around the pivot. There are many different versions of quickSort that pick the pivot in different ways.

- Always pick the first element as a pivot.
- Always pick the last element as a pivot.
- Pick a random element as a pivot.
- Pick the median as the pivot.

Notes Prepared By: - Arvind Sharma (Assistant Professor, Department of CSE, MITRC Alwar)

UNIT-3 (SORTING AND SEARCH TECHNIQUES)

HEAP SORT

LECTURE NO:-20

Heap sort is a comparison-based sorting technique based on Binary Heap data structure. It is similar to the selection sort where we first find the minimum element and place the minimum element at the beginning. Repeat the same process for the remaining elements.

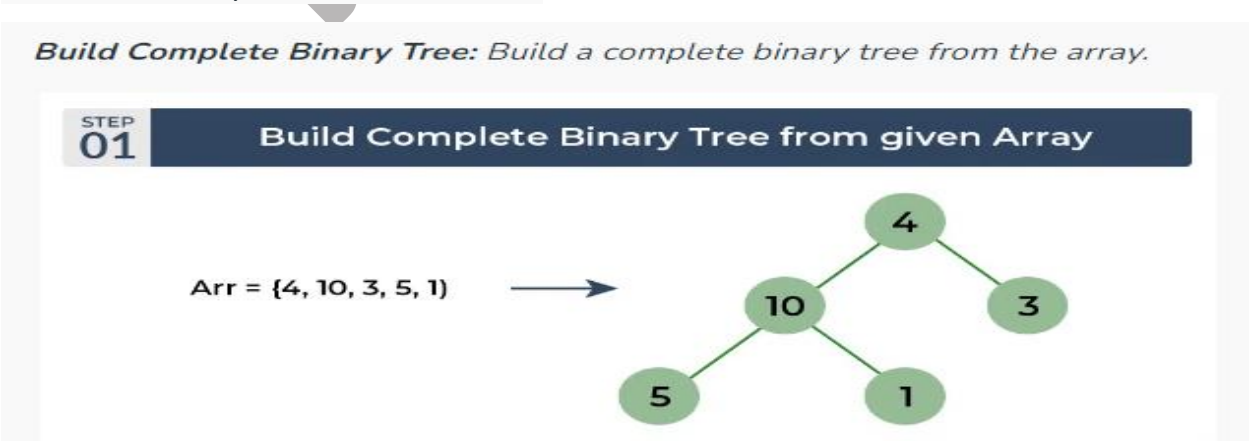
Heap Sort Algorithm

First convert the array into heap data structure using heapify, then one by one delete the root node of the Max-heap and replace it with the last node in the heap and then heapify the root of the heap. Repeat this process until size of heap is greater than 1.

- Build a heap from the given input array.
- Repeat the following steps until the heap contains only one element:
  - Swap the root element of the heap (which is the largest element) with the last element of the heap.
  - Remove the last element of the heap (which is now in the correct position).
  - Heapify the remaining elements of the heap.
- The sorted array is obtained by reversing the order of the elements in the input array.

To understand heap sort more clearly, let's take an unsorted array and try to sort it using heap sort.

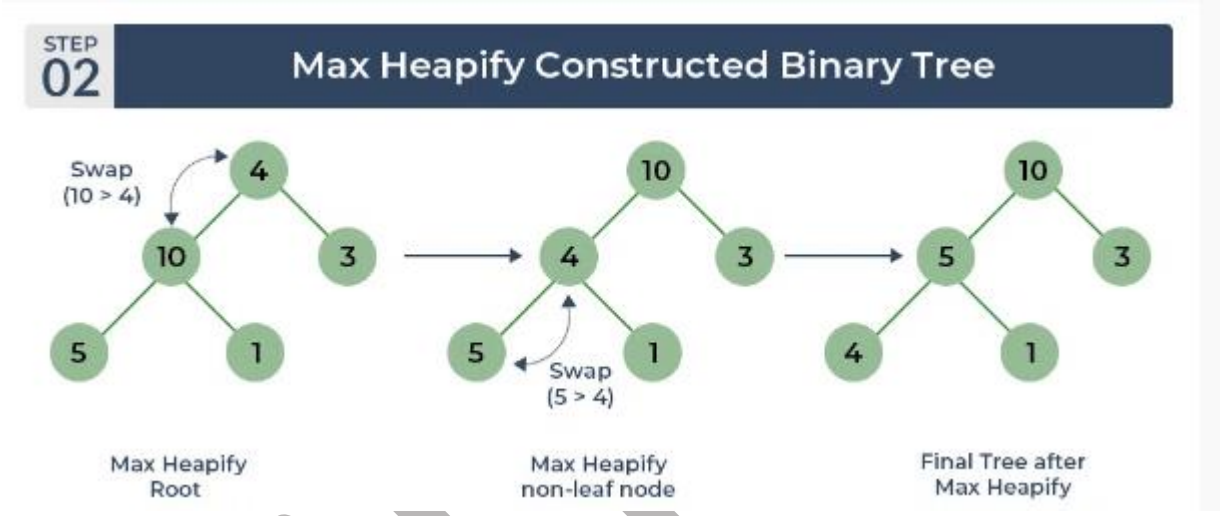
Consider the array: arr[] = {4, 10, 3, 5, 1}.



Notes Prepared By: - Arvind Sharma (Assistant Professor, Department of CSE, MITRC Alwar)

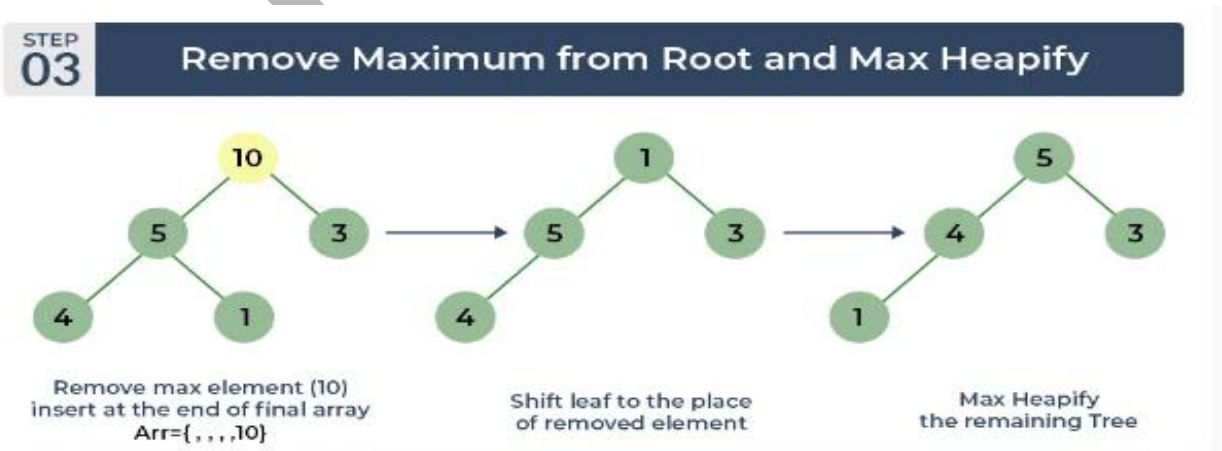
Transform into max heap: After that, the task is to construct a tree from that unsorted array and try to convert it into max heap.

- To transform a heap into a max-heap, the parent node should always be greater than or equal to the child nodes
    - Here, in this example, as the parent node **4** is smaller than the child node **10,** thus, swap them to build a max-heap.
- Now, **4** as a parent is smaller than the child **5**, thus swap both of these again and the resulted heap and array should be like this:

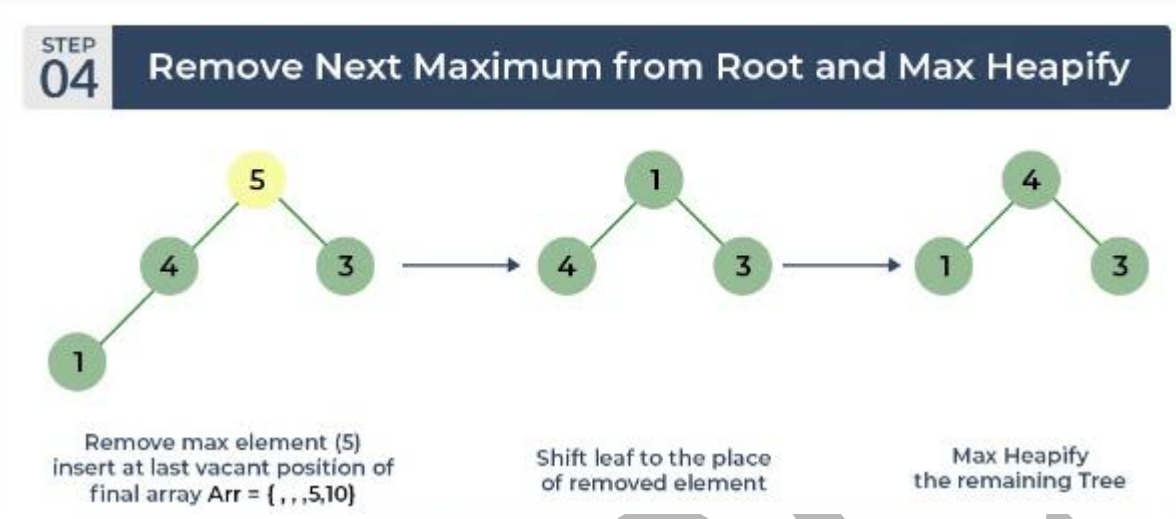


Perform heap sort: *Remove the maximum element in each step (i.e., move it to the end position and remove that) and then consider the remaining elements and transform it into a max heap.*

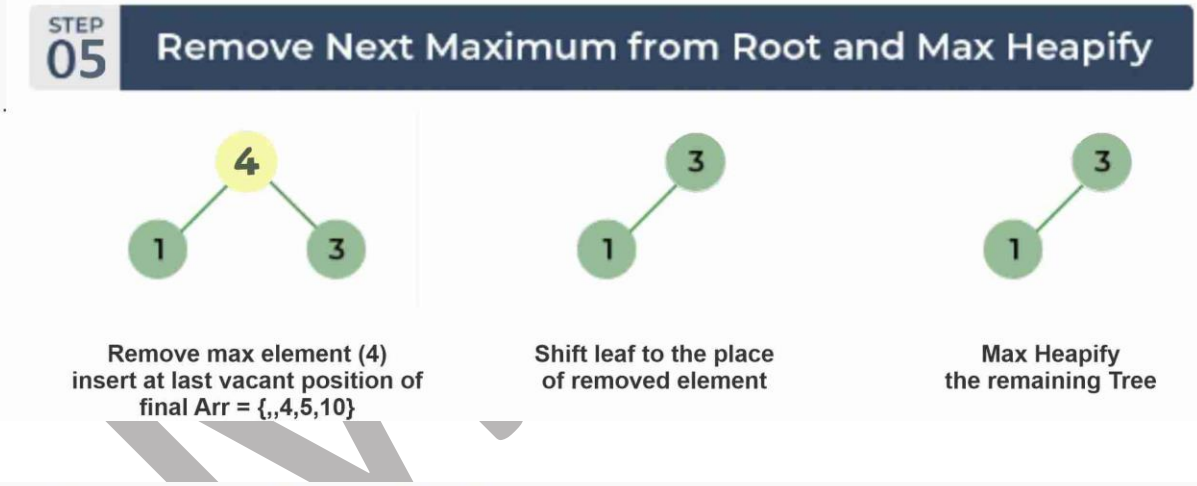
- *Delete the root element **(10)** from the max heap. In order to delete this node, try to swap it with the last node, i.e. **(1).** After removing the root element, again heapify it to convert it into max heap.*
- *Resulted heap and array should look like this:*



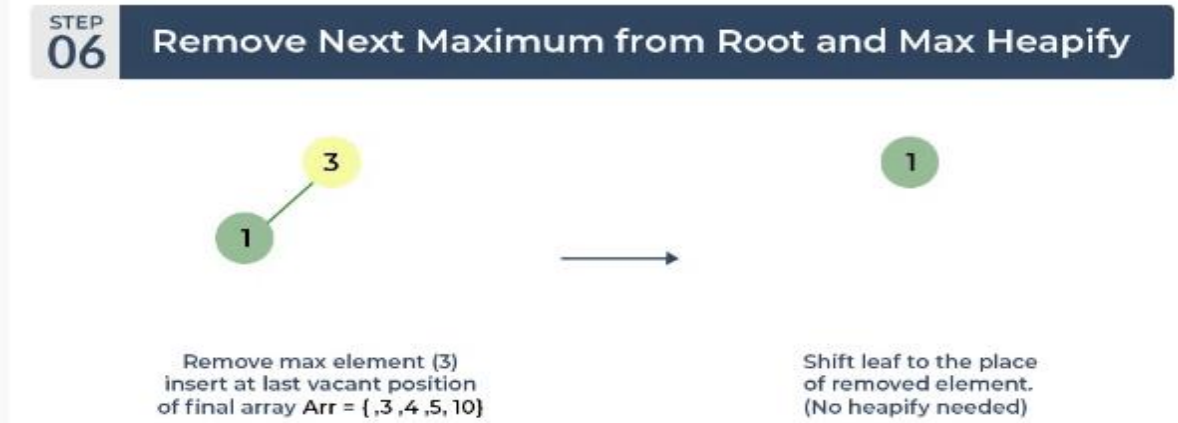
- *Repeat the above steps and it will look like the following:*



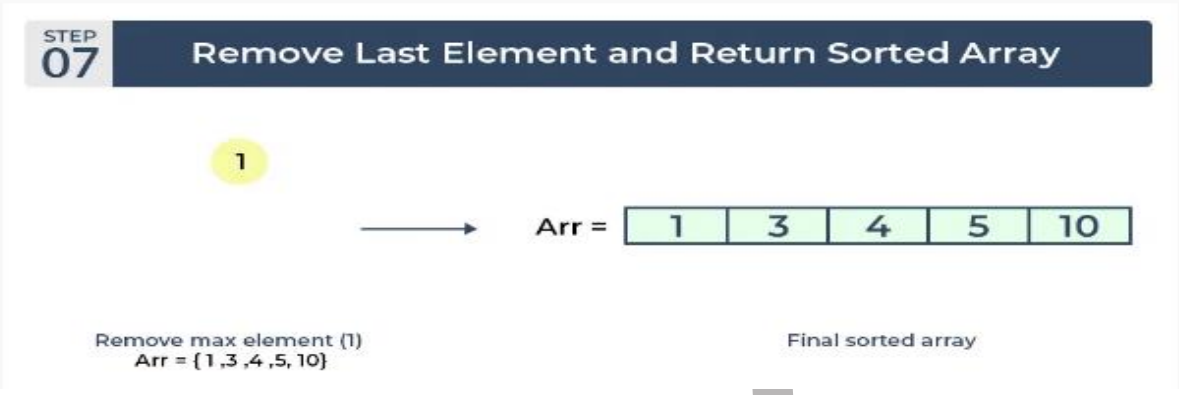
- *Now remove the root (i.e. 4) again and perform heapify*



- *Now remove the root (i.e. 3) again and perform heapify.*



- *Now when the root is removed once again it is sorted. and the sorted array will be like* ***arr[] = {1, 3, 4, 5, 10}***.



Advantages of Heap Sort:

- **Efficient Time Complexity:** Heap Sort has a time complexity of O(n log n) in all cases. This makes it efficient for sorting large datasets. The **log n** factor comes from the height of the binary heap, and it ensures that the algorithm maintains good performance even with a large number of elements.
- **Memory Usage –** Memory usage can be minimal because apart from what is necessary to hold the initial list of items to be sorted, it needs no additional memory space to work
- **Simplicity –** It is simpler to understand than other equally efficient sorting algorithms because it does not use advanced computer science concepts such as recursion.

Disadvantages of Heap Sort:

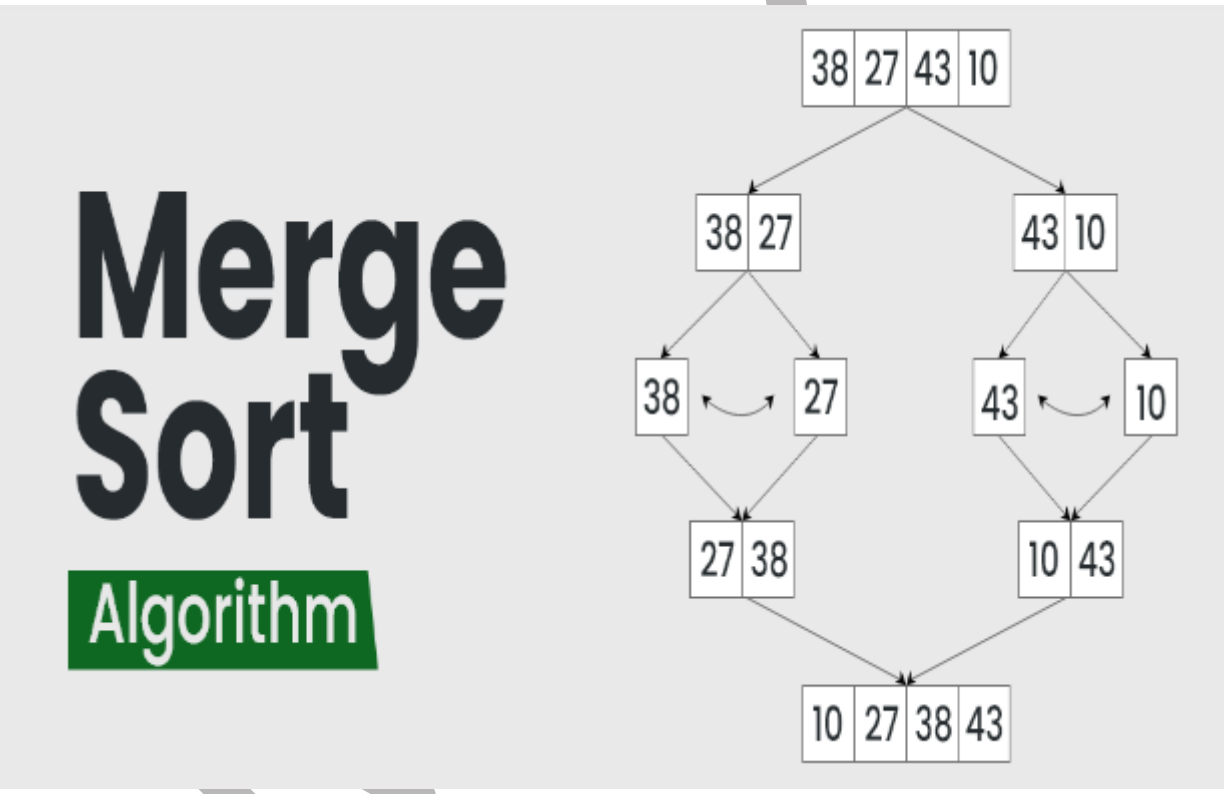
- **Costly**: Heap sort is costly.
- **Unstable**: Heap sort is unstable. It might rearrange the relative order.
- **Efficient:** Heap Sort is not very efficient when working with highly complex data.

UNIT-3 (SORTING AND SEARCH TECHNIQUES)

MERGE SORT

LECTURE NO:-21

Merge sort is defined as a sorting algorithm that works by dividing an array into smaller subarrays, sorting each subarray, and then merging the sorted subarrays back together to form the final sorted array.



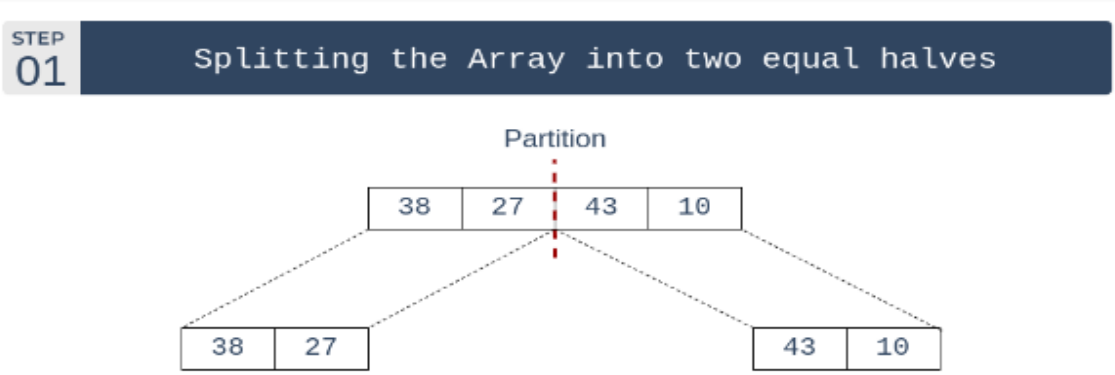
Working of Merge Sort

Merge sort is a recursive algorithm that continuously splits the array in half until it cannot be further divided i.e., the array has only one element left (an array with one element is always sorted). Then the sorted subarrays are merged into one sorted array.

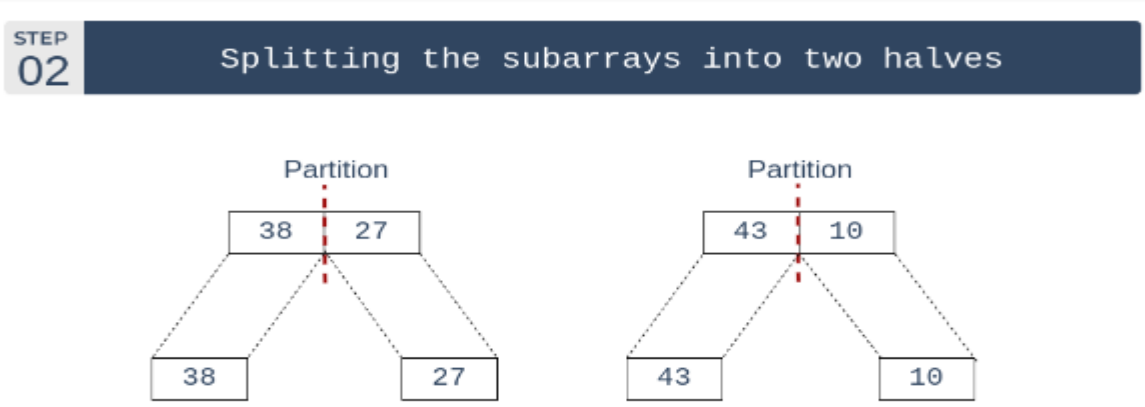
Notes Prepared By: - Arvind Sharma (Assistant Professor, Department of CSE, MITRC Alwar)

Lets consider an array ***arr[] = {38, 27, 43, 10}***

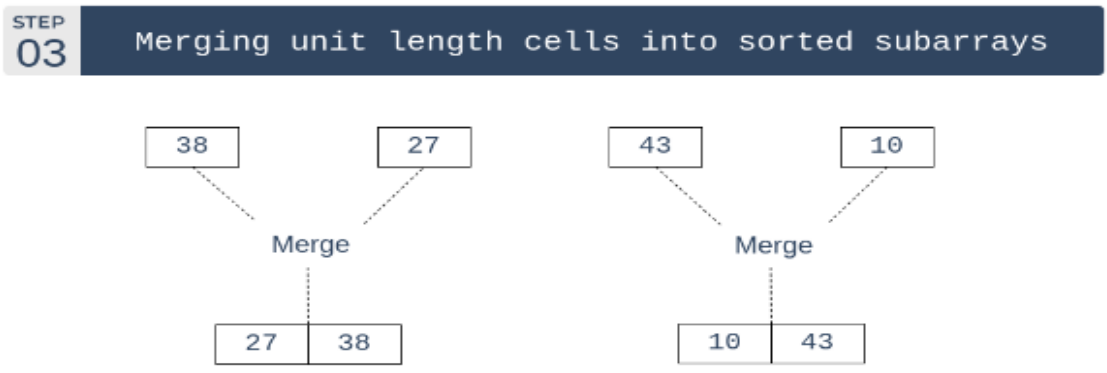
- *Initially divide the array into two equal halves:*



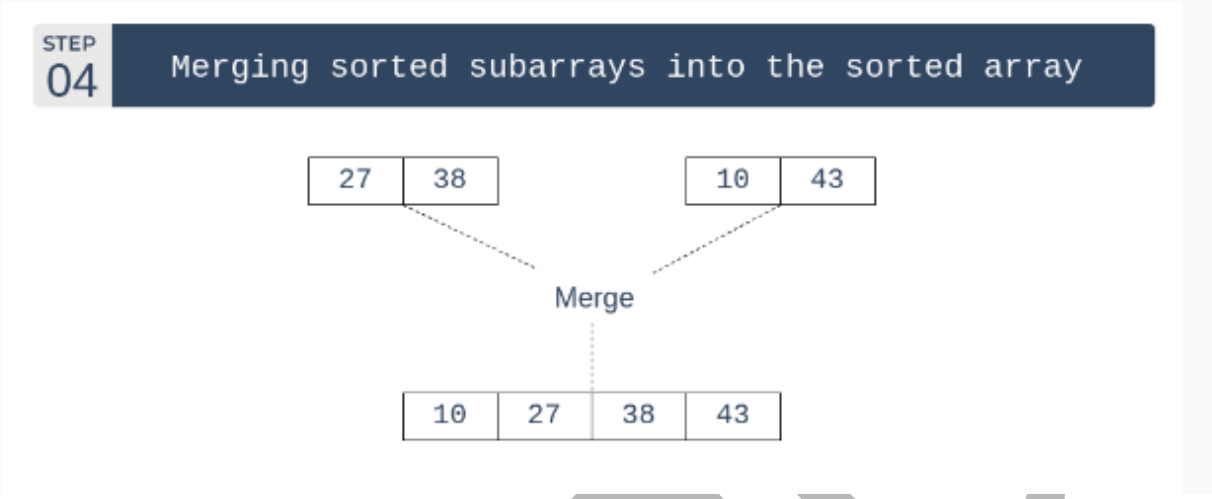
- *These subarrays are further divided into two halves. Now they become array of unit length that can no longer be divided and array of unit length are always sorted.*



These sorted subarrays are merged together, and we get bigger sorted subarrays.



This merging process is continued until the sorted array is built from the smaller subarrays.



Applications of Merge Sort:

- **Sorting large datasets:** Merge sort is particularly well-suited for sorting large datasets due to its guaranteed worst-case time complexity of O(n log n).
- **External sorting:** Merge sort is commonly used in external sorting, where the data to be sorted is too large to fit into memory.
- **Custom sorting:** Merge sort can be adapted to handle different input distributions, such as partially sorted, nearly sorted, or completely unsorted data.

Advantages of Merge Sort:

- **Stability**: Merge sort is a stable sorting algorithm, which means it maintains the relative order of equal elements in the input array.
- **Guaranteed worst-case performance:** Merge sort has a worst-case time complexity of O(N logN), which means it performs well even on large datasets.
- **Parallelizable**: Merge sort is a naturally parallelizable algorithm, which means it can be easily parallelized to take advantage of multiple processors or threads.

Drawbacks of Merge Sort:

- **Space complexity:** Merge sort requires additional memory to store the merged sub-arrays during the sorting process.

- **Not in-place:** Merge sort is not an in-place sorting algorithm, which means it requires additional memory to store the sorted data. This can be a disadvantage in applications where memory usage is a concern.
- **Not always optimal for small datasets:** For small datasets, Merge sort has a higher time complexity than some other sorting algorithms, such as insertion sort. This can result in slower performance for very small datasets.

MITRC

UNIT-3 (SORTING AND SEARCH TECHNIQUES)

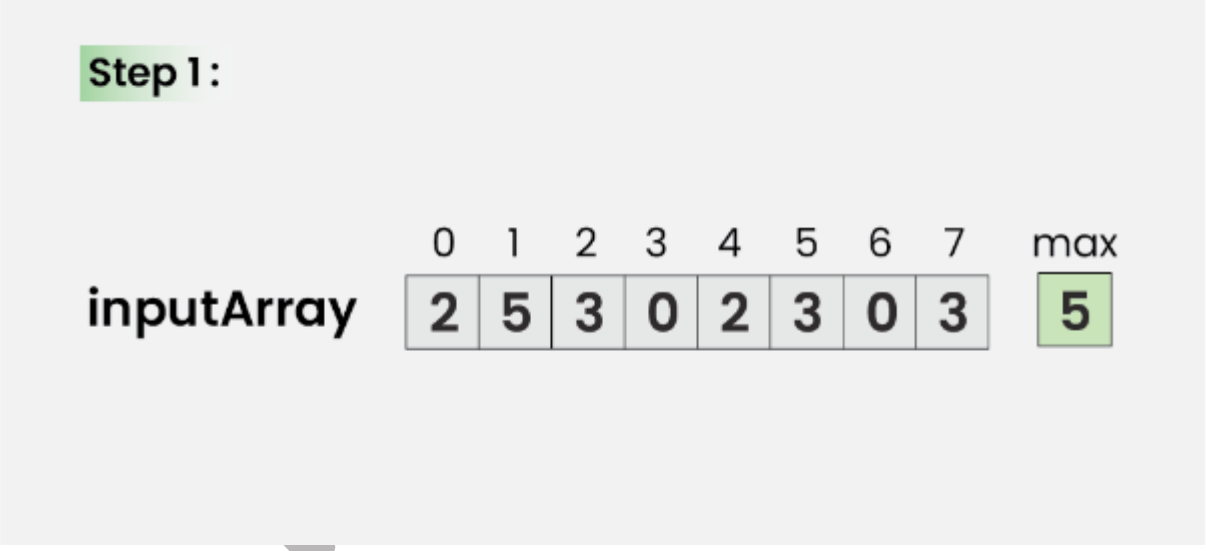
COUNTING SORT

LECTURE NO:-22

Counting Sort is a **non-comparison-based** sorting algorithm that works well when there is limited range of input values. It is particularly efficient when the range of input values is small compared to the number of elements to be sorted. The basic idea behind **Counting Sort** is to count the **frequency** of each distinct element in the input array and use that information to place the elements in their correct sorted positions.

Step1 :

- Find out the **maximum** element from the given array.



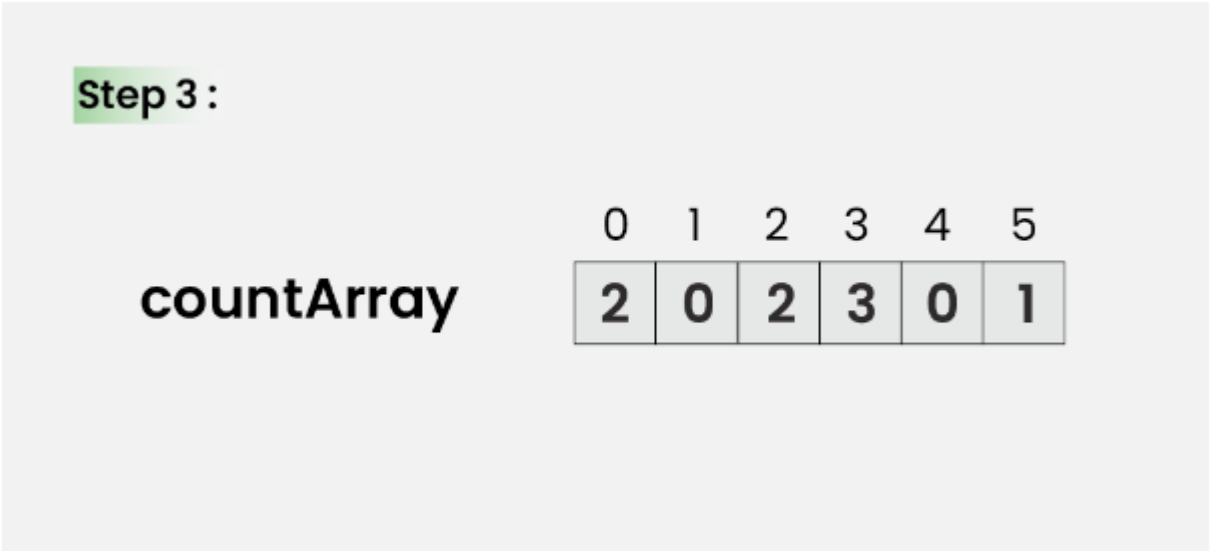
Step 2:

- Initialize a **countArray[]** of length **max+1** with all elements as **0**. This array will be used for storing the occurrences of the elements of the input array.



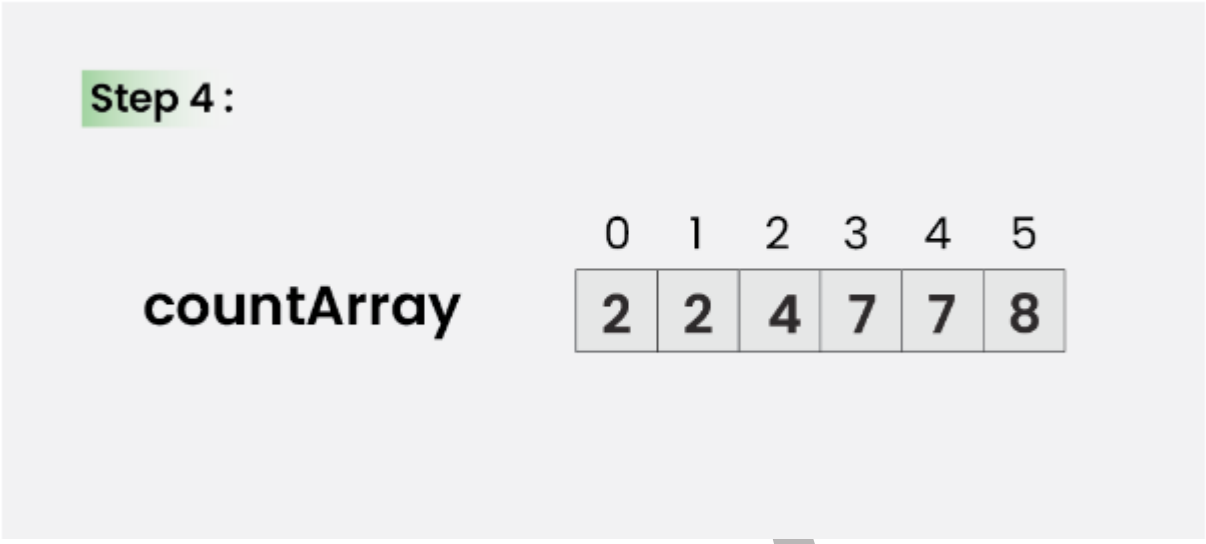
Step 3:

- In the **countArray[]**, store the count of each unique element of the input array at their respective indices.
- **For Example:** The count of element **2** in the input array is **2.** So, store **2** at index **2** in the **countArray[]**. Similarly, the count of element **5** in the input array is **1**, hence store **1** at index **5** in the **countArray[]**.



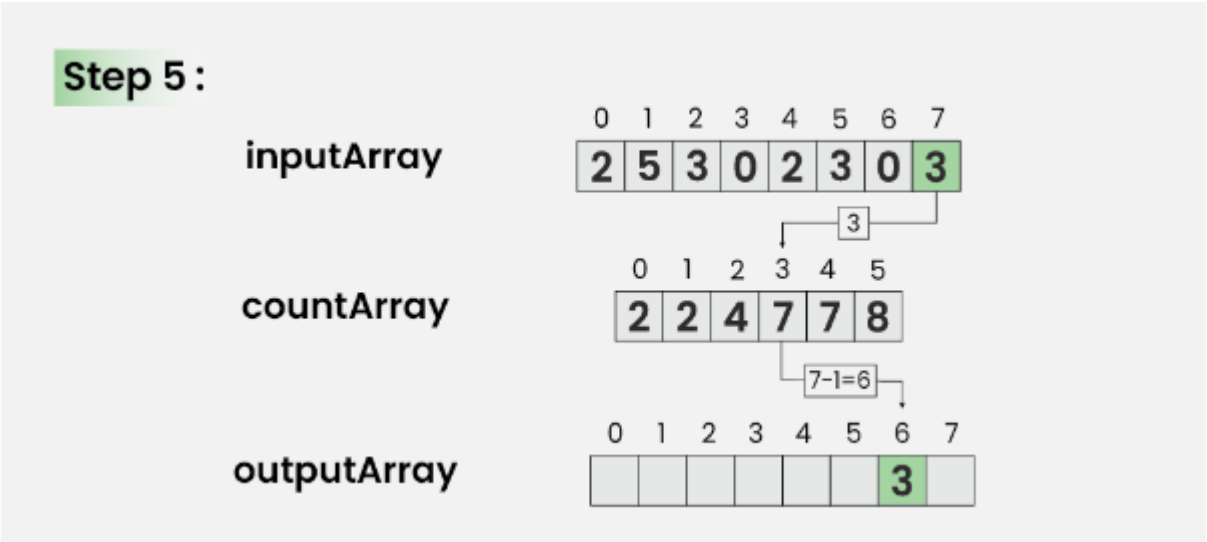
Step 4:

- Store the **cumulative sum** or **prefix sum** of the elements of the **countArray[]** by doing **countArray[i] = countArray[i – 1] + countArray[i].** This will help in placing the elements of the input array at the correct index in the output array.



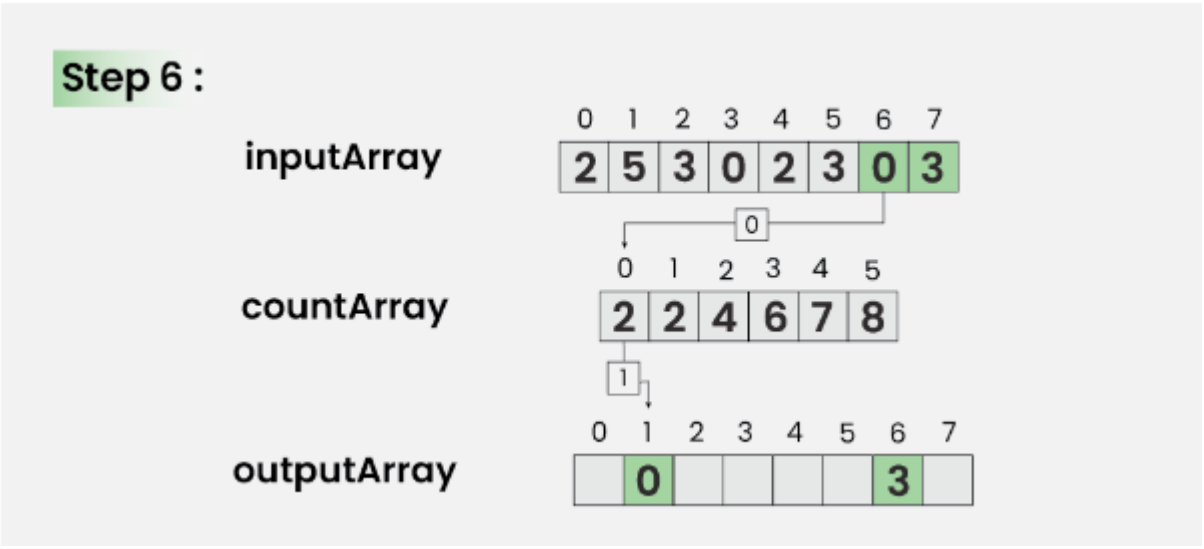
Step 5:

- Iterate from end of the input array and because traversing input array from end preserves the order of equal elements, which eventually makes this sorting algorithm **stable**.
- Update **outputArray[countArray[inputArray[i]] – 1] = inputArray[i]**. Also, update **countArray[inputArray[i]] = countArray[inputArray[i]]**



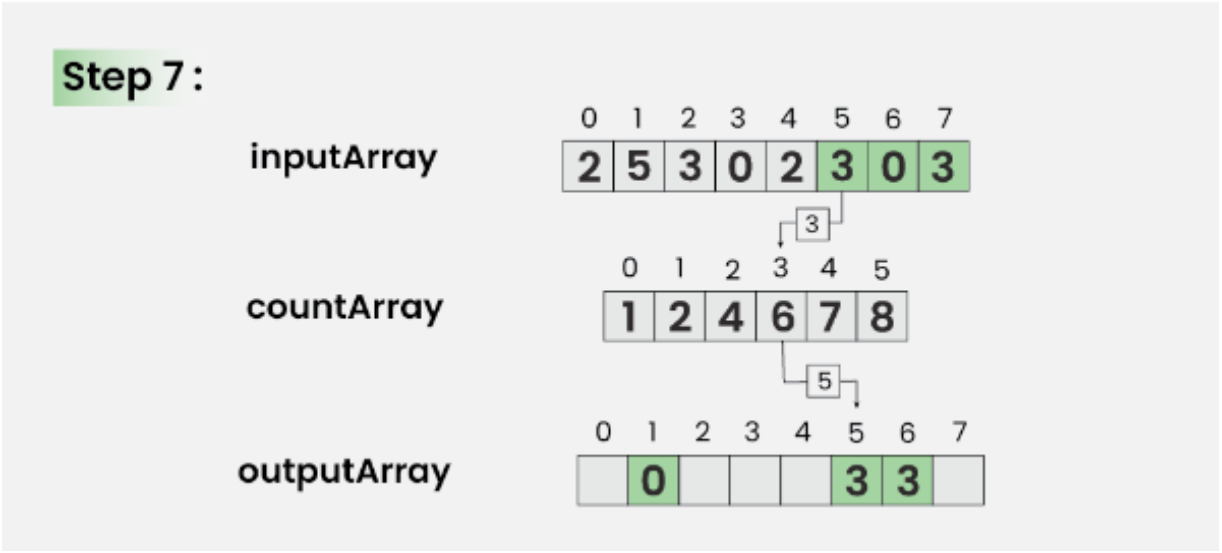
Step 6:

- **For i = 6**, Update outputArray[countArray[inputArray[6]] – 1] = inputArray[6]. Also, update countArray[inputArray[6]] = countArray[inputArray[6]]–.



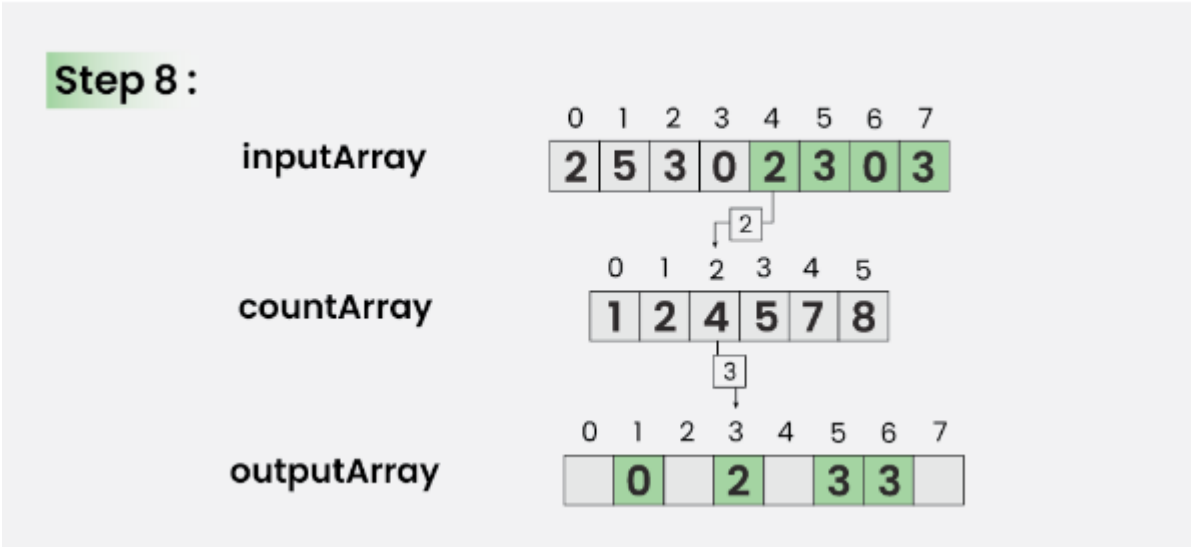
Step 7:

- **For i = 5**, Update outputArray[countArray[inputArray[5]] – 1] = inputArray[5]. Also, update countArray[inputArray[5]] = countArray[inputArray[5]]–.



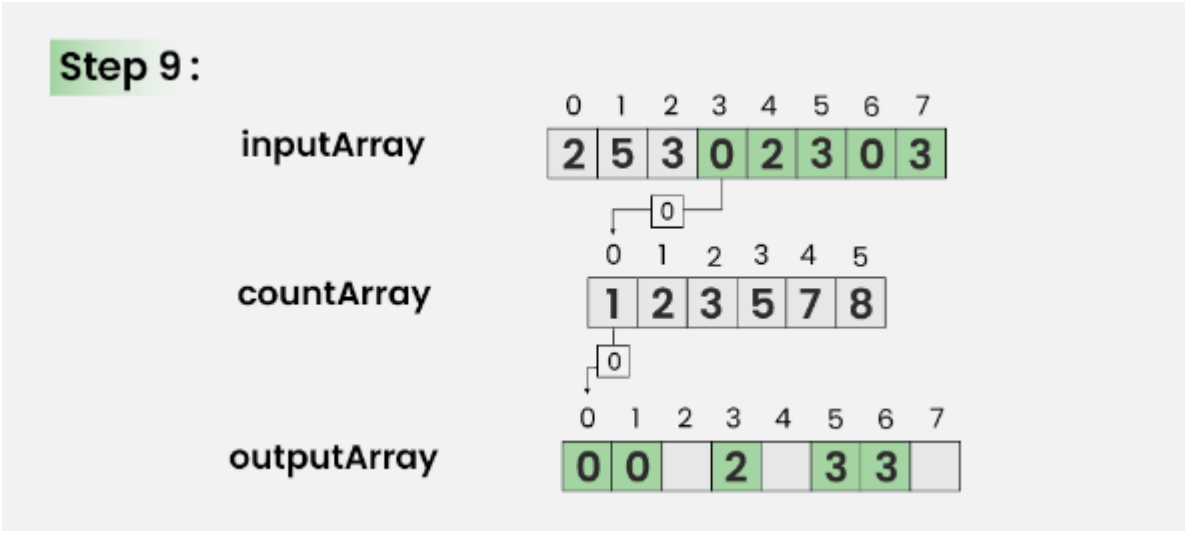
Step 8:

- **For i = 4**, Update outputArray[countArray[inputArray[4]] – 1] = inputArray[4]. Also, update countArray[inputArray[4]] = countArray[inputArray[4]]–.



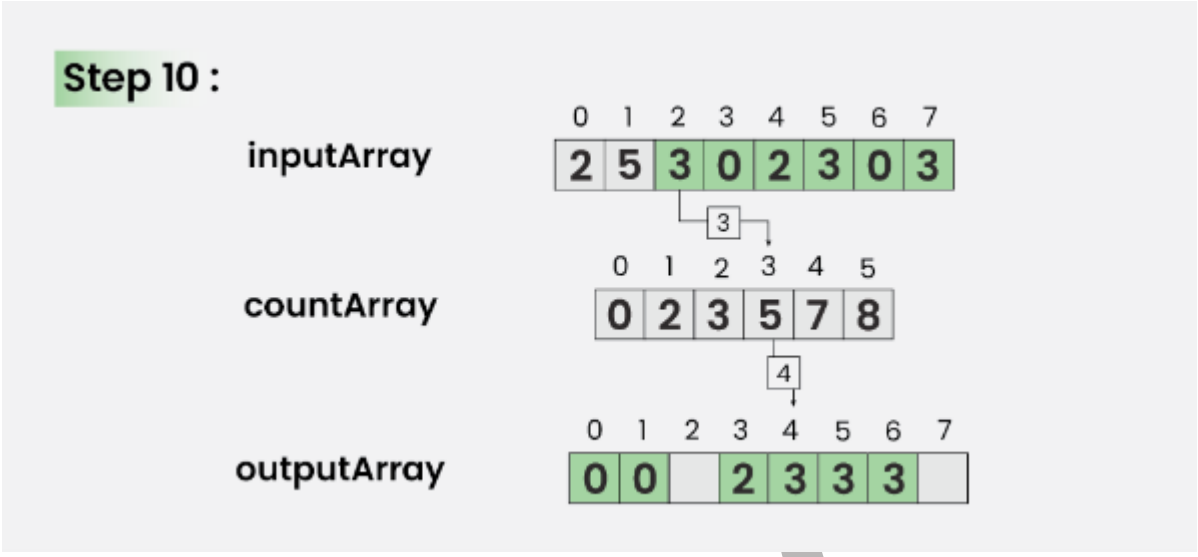
Step 9:

For i = 3, Update outputArray[countArray[inputArray[3]] – 1] = inputArray[3]. Also, update countArray[inputArray[3]] = countArray[inputArray[3]]



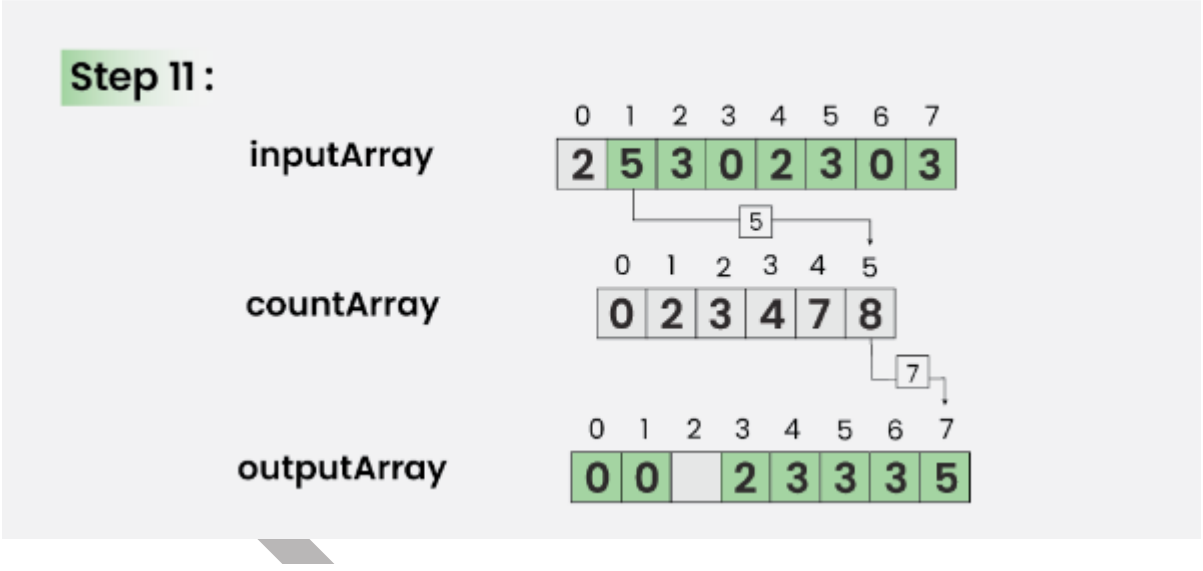
Step 10:

For i = 2, Update outputArray[countArray[inputArray[2]] – 1] = inputArray[2]. Also, update countArray[inputArray[2]] = countArray[inputArray[2]]–.



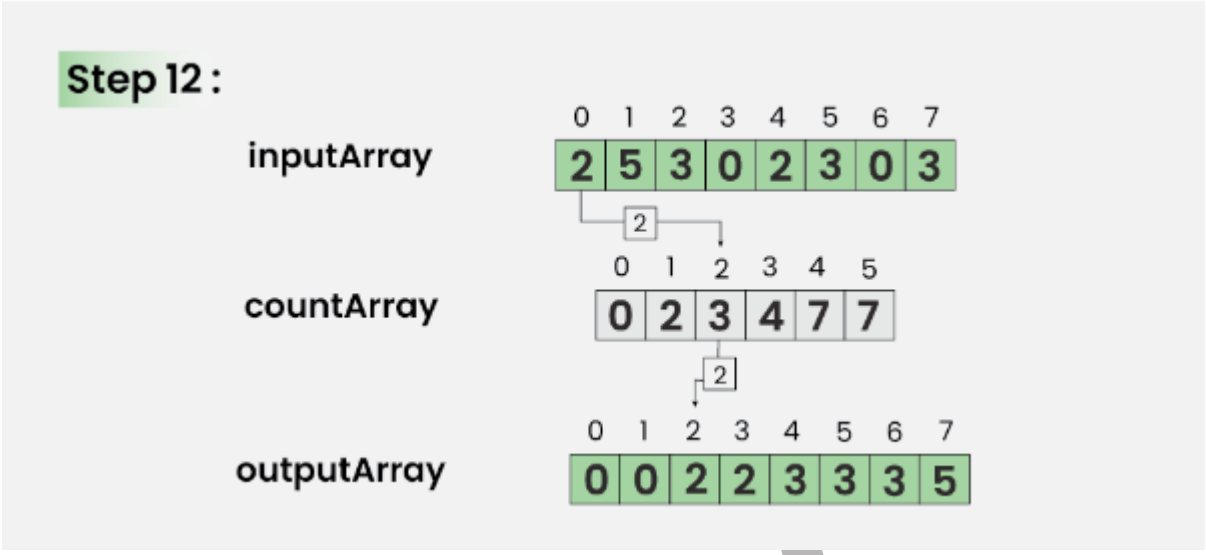
Step 11:

For i = 1, Update outputArray[countArray[inputArray[1]] – 1] = inputArray[1]. Also, update countArray[inputArray[1]] = countArray[inputArray[1]]–.



Step 12:

For i = 0, Update outputArray[countArray[inputArray[0]] – 1] = inputArray[0]. Also, update countArray[inputArray[0]] = countArray[inputArray[0]]–.



Counting Sort Algorithm

- Declare an auxiliary array **countArray[]** of size **max(inputArray[])+1** and initialize it with **0**.
- Traverse array **inputArray[]** and map each element of **inputArray[]** as an index of **countArray[]** array, i.e., execute **countArray[inputArray[i]]++** for **0 <= i < N**.
- Calculate the prefix sum at every index of array **inputArray**[].
- Create an array **outputArray[]** of size **N**.
- Traverse array **inputArray[]** from end and update **outputArray[countArray[inputArray[i]] – 1] = inputArray[i]**. Also, update **countArray[inputArray[i]] = countArray[inputArray[i]]–**.

Advantages of Counting Sort

- Counting sort generally performs faster than all comparison-based sorting algorithms, such as merge sort and quicksort, if the range of input is of the order of the number of input.
- Counting sort is easy to code
- Counting sort is a **stable algorithm**.

Disadvantages of Counting Sort

- Counting sort doesn't work on decimal values.
- Counting sort is inefficient if the range of values to be sorted is very large.
- Counting sort is not an **In-place sorting** algorithm, It uses extra space for sorting the array elements.

MITRC

UNIT-3 (SORTING AND SEARCH TECHNIQUES)

EXTERNAL SORTING, INTERNAL SORTING, STABLE & UNSTABLE SORTING

LECTURE NO:-23

External sorting is a term for a class of sorting algorithms that can handle massive amounts of data. External sorting is required when the data being sorted does not fit into the main memory of a computing device (usually RAM) and instead, must reside in the slower external memory (usually a hard drive).

External sorting typically uses a hybrid sort-merge strategy. In the sorting phase, chunks of data small enough to fit in the main memory are read, sorted, and written out to a temporary file. In the merge phase, the sorted sub-files are combined into a single larger file.

Internal Sorting algorithms that use main memory exclusively during the sort are called internal sorting algorithms. This kind of algorithm assumes high-speed random access to all memory. Some of the common algorithms that use this sorting feature are : Bubble Sort, Insertion Sort., and Quick Sort.

In a **stable sorting** algorithm, data is sorted in a way that preserves the original order of elements having equal keys. This means that if two elements have the same key, the one that appeared earlier in the input will also appear earlier in the sorted output. For instance, if element A[i] and element A[j] have the same key and i < j, then in sorted order, A[i] should come before A[j]. Example are : **Merge sort, Insertion sort, Bubble sort, Counting sort, Radix sort:**

In an **unstable sorting** algorithm, data is sorted so that the order of elements with equal keys is not preserved after sorting. For instance, if element A[i] and element A[j] have the same key and i < j, then in sorted order, there is a possibility that A[j] can come before A[i]. Examples are : **Quick sort, Heap sort, Shell sort.**

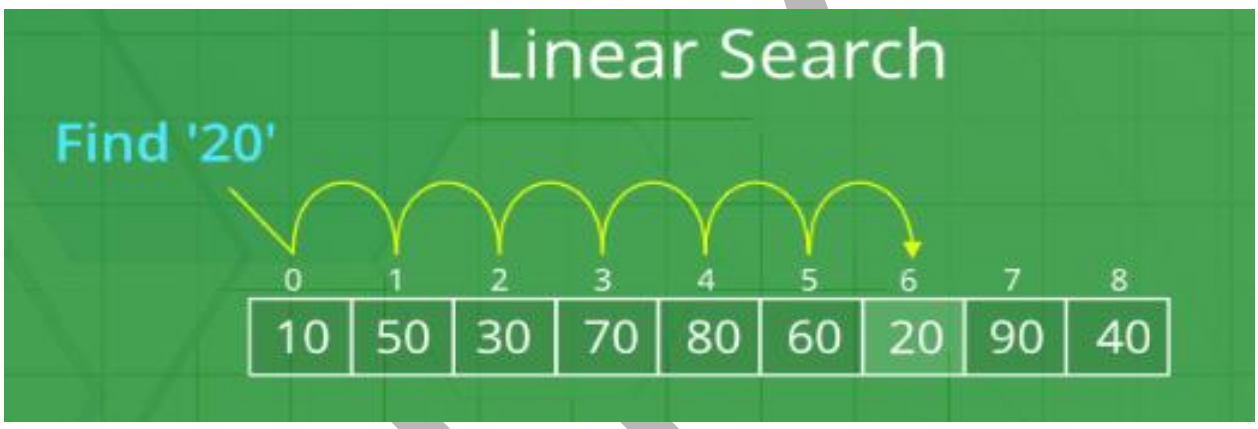
Notes Prepared By: - Arvind Sharma (Assistant Professor, Department of CSE, MITRC Alwar)

UNIT-3 (SORTING AND SEARCH TECHNIQUES)

SEARCHING: LINEAR SEARCH

LECTURE NO:-24

Linear Search is defined as a sequential search algorithm that starts at one end and goes through each element of a list until the desired element is found, otherwise the search continues till the end of the data set.



In Linear Search Algorithm,

- Every element is considered as a potential match for the key and checked for the same.
- If any element is found equal to the key, the search is successful and the index of that element is returned.
- If no element is found equal to the key, the search yields “No match found”.

For example: Consider the array **arr[] = {10, 50, 30, 70, 80, 20, 90, 40}** and **key** = 30

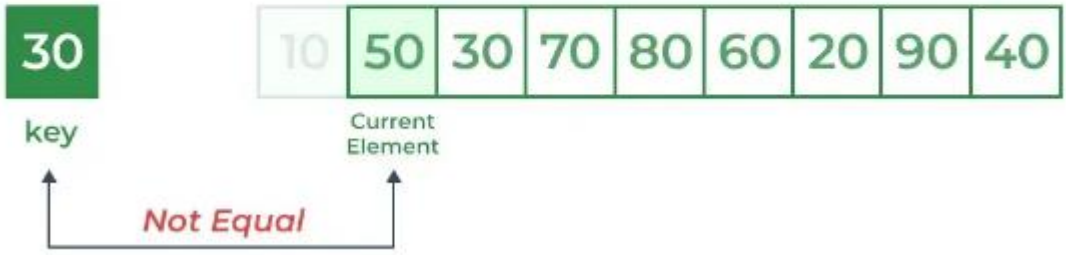
Step 1: *Start from the first element (index 0) and compare **key** with each element (arr[i]).*

- *Comparing key with first element arr[0]. SInce not equal, the iterator moves to the next element as a potential match.*

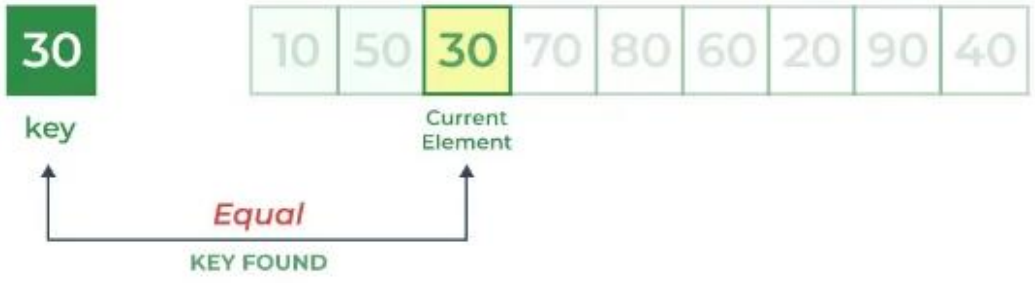
Notes Prepared By: - Arvind Sharma (Assistant Professor, Department of CSE, MITRC Alwar)



- *Comparing key with next element arr[1]. SInce not equal, the iterator moves to the next element as a potential match.*



Step 2: *Now when comparing arr[2] with key, the value matches. So the Linear Search Algorithm will yield a successful message and return the index of the element when key is found (here 2).*



Advantages of Linear Search

- Linear search can be used irrespective of whether the array is sorted or not. It can be used on arrays of any data type.
- Does not require any additional memory.
- It is a well-suited algorithm for small datasets.

Drawbacks of Linear Search

- Linear search has a time complexity of O(N), which in turn makes it slow for large datasets.
- Not suitable for large arrays.

When to use Linear Search

- When we are dealing with a small dataset.
- When you are searching for a dataset stored in contiguous memory.

The procedures for implementing linear search are as follows:

Step 1: First, read the search element (Target element) in the array.

Step 2: In the second step compare the search element with the first element in the array.

Step 3: If both are matched, display "Target element is found" and terminate the Linear Search function.

Step 4: If both are not matched, compare the search element with the next element in the array.

Step 5: In this step, repeat steps 3 and 4 until the search (Target) element is compared with the last element of the array.

Step 6: If the last element in the list does not match, the Linear Search Function will be terminated, and the message "Element is not found" will be displayed.

UNIT-3 (SORTING AND SEARCH TECHNIQUES)

BINARY SEARCH

LECTURE NO:-25

Binary Search is defined as a searching algorithm used in a sorted array by **repeatedly dividing the search interval in half**. The idea of binary search is to use the information that the array is sorted and reduce the time complexity to O(log N).



To apply Binary Search algorithm:

- The data structure must be sorted.
- Access to any element of the data structure takes constant time.

Binary Search Algorithm:

- *Divide the search space into two halves by* finding the middle index "mid".
- *Compare the middle element of the search space with the key.*
- *If the key is found at middle element, the process is terminated.*
- *If the key is not found at middle element, choose which half will be used as the next search space.*

- *If the key is smaller than the middle element, then the left side is used for next search.*
- *If the key is larger than the middle element, then the right side is used for next search.*

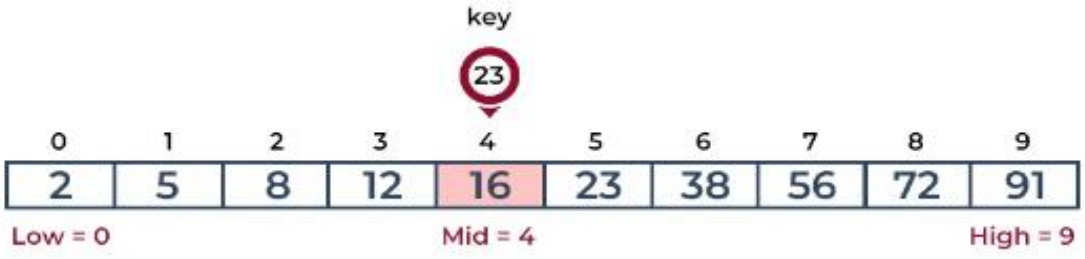
- *This process is continued until the key is found or the total search space is exhausted.*

To understand the working of binary search, consider the following illustration:

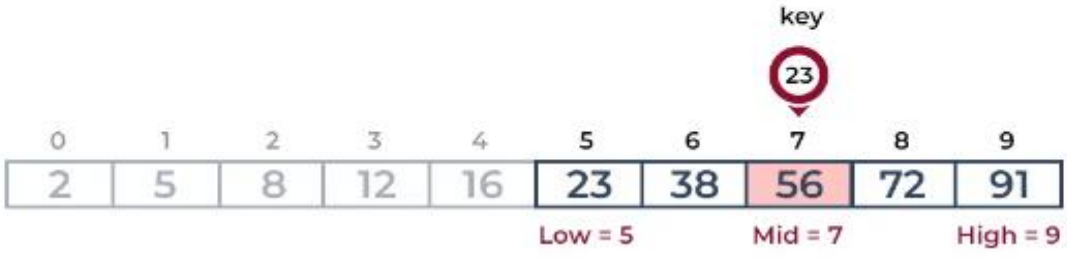
*Consider an array **arr[] = {2, 5, 8, 12, 16, 23, 38, 56, 72, 91}**, and the **target = 23**.*

***First Step:** Calculate the mid and compare the mid element with the key. If the key is less than mid element, move to left and if it is greater than the mid then move search space to the right.*

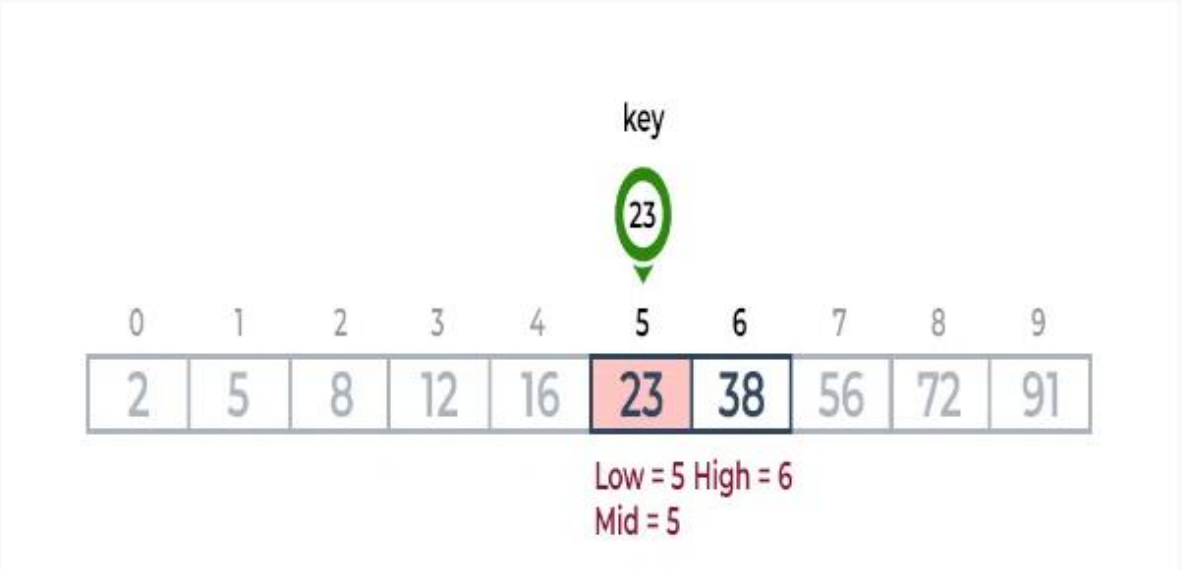
- *Key (i.e., 23) is greater than current mid element (i.e., 16). The search space moves to the right.*



- *Key is less than the current mid 56. The search space moves to the left.*



Second Step: *If the key matches the value of the mid element, the element is found and stop search.*



The **Binary Search Algorithm** can be implemented in the following two ways

- Iterative Binary Search Algorithm
- Recursive Binary Search Algorithm